

Relazione Tecnica di Progetto

Wiki Collaborativo per Appunti

Domenico Aloï

Anno Accademico 2025/26 - Ingegneria del Software

Indice

1	Introduzione	3
1.1	As-Is	3
1.2	To-Be	3
2	Requisiti	5
2.1	Requisiti Funzionali (RF)	5
2.2	Requisiti Non Funzionali (RNF)	6
3	Glossario	8
4	Diagramma dei casi d'uso	10
4.1	Dettagli del Caso d'Uso UC1	11
4.2	Dettagli del Caso d'Uso UC2	14
4.3	Dettagli del Caso d'Uso UC3	18
4.4	Matrice di tracciabilità	20
5	Progettazione	21
5.1	Diagramma delle Classi FE	21
5.2	Diagramma delle Classi BE	22
5.2.1	Diagramma delle Classi Entry Point	22
5.2.2	Diagramma delle Classi Gateway	22
5.2.3	Diagramma delle Classi Strategy	23
5.3	Diagrammi di Sequenza	23
5.3.1	Diagramma di Sequenza: Login (RF2)	24
5.3.2	Diagramma di Sequenza: Ricerca Appunti (RF9)	26
5.4	Diagrammi di Attività	27
5.4.1	UC1 - Diagramma delle Attività: RF5 - Visualizzazio- ne Dettaglio	27
5.4.2	UC1 - Diagramma delle Attività: RF9 - Ricerca Appunti	28
5.5	Progettazione dei Dati	29
5.6	Specifica REST API	29

5.6.1	Endpoint	30
6	Architettura	36
6.1	Design Pattern Model-View-Presenter (MVP)	37
6.1.1	Mapping nel codice	37
6.2	Design Pattern Table Data Gateway	37
6.2.1	Mapping nel codice	38
6.3	Design Pattern Strategy	39
6.3.1	Mapping nel codice	39
6.4	Design Pattern Memento	40
6.4.1	Mapping nel codice	40
7	Manuale Tecnico	41
7.1	Introduzione	41
7.2	Requisiti di Sistema	41
7.3	Inizializzazione del Database	41
7.3.1	Logica di Popolamento	42
7.4	Script di Automazione	42
7.4.1	Avvio del Progetto (<code>run_project.sh</code>)	42
7.4.2	Arresto del Progetto (<code>stop_project.sh</code>)	42
7.4.3	Reset Totale (<code>reset_all.sh</code>)	42
7.5	Procedura di Installazione Standard	43
7.6	Credenziali di Test	43
7.7	Risoluzione dei Problemi	43

Capitolo 1

Introduzione

La richiesta consiste nel creare un sistema dove gli studenti possono caricare, modificare e versionare appunti per i corsi. Principalmente si tratterà di un editor di testo (semplice markdown o text-area) dotato di cronologia delle versioni (semplificata) e un'organizzazione per corso/argomento.

1.1 As-Is

L'attuale gestione e condivisione degli appunti tra gli studenti universitari avviene prevalentemente in modo frammentato e manuale. Gli appunti sono dispersi su diverse piattaforme (email, chat, cloud personali), rendendo difficile la reperibilità del materiale. Non avendo un sistema centralizzato è altamente probabile disperdere correzioni o aggiornamenti importanti. In questo contesto la collaborazione risulta molto complicata e limitata all'iniziativa personale.

1.2 To-Be

Il sistema **Wiki Collaborativo per Appunti** si propone di risolvere tali problematiche attraverso un sistema centralizzato di collaborazione tra gli studenti. Esso infatti permetterà di:

- **Condividere il materiale:** Gli appunti sono condivisi in un unico sistema alla portata di tutti e a disposizione di tutti.
- **Collaborare:** Ogni utente, dopo la registrazione, può contribuire aggiungendo nuovi appunti o modificando gli esistenti.

- **Controllare il materiale:** Ogni modifica agli appunti genera automaticamente una nuova versione. Questo consente agli utenti di navigare nella cronologia e, in caso di errori rilevati, ripristinare la vecchia versione di un appunto.

Capitolo 2

Requisiti

2.1 Requisiti Funzionali (RF)

RF1 Registrazione utente

- L'utente può creare un account fornendo: nome, email e password.
- Non è richiesto un sistema di validazione dell'email.

RF2 Login/Logout

- L'utente può accedere al sistema tramite credenziali e terminare la sessione (logout).

RF3 Visualizzazione corsi

- Qualsiasi utente (autenticato o visitatore) può visualizzare l'elenco dei corsi disponibili.

RF4 Visualizzazione argomenti di un corso

- Ogni corso presenta una lista di argomenti.
- L'utente può navigare agevolmente tra i corsi e i relativi argomenti.

RF5 Visualizzazione degli appunti

- L'utente può visualizzare gli appunti associati a un argomento.
- Ogni appunto è leggibile; il contenuto è visualizzato in formato HTML renderizzato (da sorgente Markdown o testo semplice).

RF6 Creazione appunti

- Gli utenti autenticati possono creare nuovi appunti.

- Inserimento obbligatorio di titolo, testo e selezione del corso/argomento di riferimento.

RF7 Modifica appunti

- Gli utenti autenticati hanno il permesso di modificare gli appunti esistenti.
- Ogni operazione di modifica genera automaticamente una nuova versione dell'appunto.

RF8 Cronologia delle versioni

- Visualizzazione dell'elenco delle versioni precedenti per ogni appunto (inclusi timestamp e autore).
- Possibilità di consultare una versione specifica del passato.
- Funzionalità di ripristino: una versione precedente può essere promossa a versione attuale.

RF9 Ricerca

- Sistema di ricerca per individuare appunti tramite il titolo.

RF10 Gestione corsi, argomenti e appunti

- Gli utenti con privilegi di amministratore possono creare, modificare o eliminare corsi, argomenti e appunti.

2.2 Requisiti Non Funzionali (RNF)

RNF1 Architettura

- L'applicazione deve essere sviluppata seguendo il pattern architetturale Model–View–Presenter (MVP).
- Frontend e Backend devono essere completamente disaccoppiati e comunicare esclusivamente tramite API.

RNF2 Tecnologie

- Frontend: utilizzo di tecnologie standard (HTML5, CSS3, JavaScript). È consentito l'uso di Bootstrap o jQuery.
- Backend: sviluppo in PHP puro (senza l'ausilio di framework come Laravel o Symfony).
- Interfaccia API: implementazione di servizi RESTful per le operazioni CRUD.
- Divieto di Server-Side Rendering (SSR): il backend non deve generare codice HTML.

- RNF3 Indipendenza Frontend/Backend**
- Il frontend deve essere progettato per poter interagire con backend alternativi (es. Java, Go).
 - Il backend deve poter servire client differenti, inclusi dispositivi mobile o applicazioni desktop.
- RNF4 Sicurezza**
- Tutte le API che comportano la modifica dei dati devono essere protette tramite token di autenticazione o gestione della sessione.
 - Le password devono essere memorizzate nel database esclusivamente tramite algoritmi di hashing sicuri.
- RNF5 Usabilità**
- L'interfaccia utente deve seguire un design semplice, pulito e minimalista.
 - Il sistema deve essere *responsive*, garantendo il supporto ai dispositivi mobili.
- RNF6 Prestazioni**
- Il tempo di risposta del sistema per le operazioni principali deve essere inferiore ai 5 secondi.
- RNF7 Documentazione**
- Il repository GitHub del progetto deve includere: requisiti, diagrammi UML, guida all'installazione, manuale utente sintetico e note di implementazione.
- RNF8 Versionamento**
- Utilizzo rigoroso di Git e GitHub: gestione dei branch, commit regolari e descrittivi.

Capitolo 3

Glossario

Tabella 3.1: Glossario dei termini principali del sistema Wiki Collaborativo

Termine	Definizione
Amministratore	Utente con privilegi elevati incaricato della gestione strutturale di corsi e argomenti. <i>Sinonimi: Admin.</i>
Appunto	L'unità informativa principale del sistema, contenente testo in formato Markdown o plain text renderizzato in HTML.
Argomento	Sotto-categoria tematica di un corso a cui vengono associati uno o più appunti.
Corso	Categoria tematica a cui vengono associati uno o più argomenti.
CRUD	Acronimo di Create, Read, Update, Delete; rappresenta le quattro operazioni di base gestite dalle API REST del sistema.
Markdown	Linguaggio di markup leggero utilizzato dagli studenti per la formattazione del testo degli appunti.
MVP	Pattern architetturale (Model-View-Presenter) che separa la logica di business dalla visualizzazione, come richiesto dai requisiti di progetto.
REST API	Interfaccia programmabile stateless che permette la comunicazione tramite protocollo HTTP tra il frontend (JS) e il backend (PHP).
Studente Autenticato	Utente registrato che ha effettuato l'accesso e possiede i permessi per creare o modificare appunti e visualizzare la cronologia. <i>Sinonimi: StudAuth.</i>
Studente	Utente non registrato che non ha effettuato l'accesso. Può solo visualizzare e cercare gli appunti senza poterli creare o modificare. <i>Sinonimi: Stud.</i>

Capitolo 4

Diagramma dei casi d'uso

Il rettangolo nel diagramma rappresenta sempre il sistema **Wiki Collaborativo per Appunti**. Ho diviso gli use case dal punto di vista degli attori:

- **Studente**: si intende come non autenticato
- **Studente Autenticato**: può fare tutto ciò che fa Studente ma ha delle funzionalità in più
- **Amministratore**: gestisce la struttura dei corsi

Ho diviso i diagrammi per attore per rendere più chiara la separazione delle responsabilità e dei permessi di accesso

4.1 Dettagli del Caso d'Uso UC1

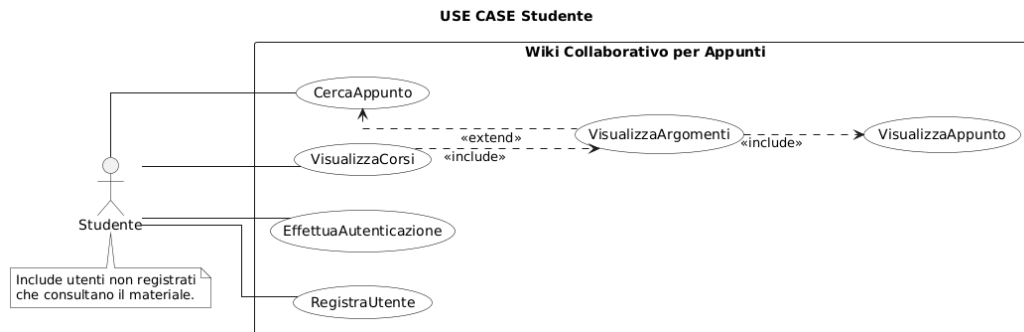


Figura 4.1: UC1 - Diagramma dei casi d'uso per l'attore Studente non autenticato

Tabella 4.1: Specifica del caso d'uso per RF1: RegistraUtente

ID	1
Nome	RegistraUtente
Descrizione	Permette a un nuovo utente di creare un account fornendo nome, email e password.
Attori	Studente.
Precondizioni	L'utente non è registrato.
Flusso Principale	1. L'utente seleziona la funzione di registrazione. 2. L'utente inserisce nome, email e password. 2. Il sistema valida il formato dei dati (es. email valida). 4. Il sistema esegue l'hashing della password per sicurezza. 5. Il sistema memorizza i dati nel database.
Postcondizioni	Un nuovo account è stato creato con successo.
Flussi Alternativi	A1. Email già presente: Il sistema informa l'utente che l'email è già registrata e interrompe la procedura.

Tabella 4.2: Specifica del caso d'uso per RF2: EffettuaAutenticazione

ID	2
Nome	EffettuaAutenticazione
Descrizione	Consente all'utente di accedere al sistema tramite le proprie credenziali.
Attori	Studente.
Precondizioni	L'utente deve possedere un account registrato.
Flusso Principale	1. L'utente accede alla funzione di login. 2. L'utente inserisce email e password. 3. Il sistema confronta le credenziali con quelle nel database. 4. Il sistema avvia una sessione autenticata.
Postcondizioni	L'utente è autenticato e può accedere alle funzioni di creazione/modifica.
Flussi Alternativi	A1. Credenziali errate: Il sistema nega l'accesso e mostra un messaggio di errore.

Tabella 4.3: Specifica del caso d'uso per RF3: VisualizzaCorsi

ID	3
Nome	VisualizzaCorsi
Descrizione	Permette di consultare l'elenco completo dei corsi disponibili nel Wiki.
Attori	Studente.
Precondizioni	Il sistema è accessibile.
Flusso Principale	1. L'utente accede alla pagina principale. 2. In automatico il sistema recupera i nomi dei corsi dal database. 3. Il sistema mostra l'elenco dei corsi all'utente.
Postcondizioni	L'utente visualizza l'elenco dei corsi.

Tabella 4.4: Specifica del caso d'uso per RF4: VisualizzaAppunti

ID	4
Nome	VisualizzaArgomenti
Descrizione	Permette di navigare tra gli argomenti associati a un corso specifico.
Attori	Studente.
Precondizioni	L'utente ha selezionato un corso (3).
Flusso Principale	1. L'utente seleziona un corso. 2. Il sistema recupera gli argomenti legati a quel corso. 3. Il sistema mostra la lista degli argomenti di quel corso.
Postcondizioni	L'utente visualizza gli appunti collegati a quell'argomento.

Tabella 4.5: Specifica del caso d'uso per RF5: VisualizzaAppunto

ID	5
Nome	VisualizzaAppunto
Descrizione	Mostra il contenuto di un appunto in formato HTML renderizzato.
Attori	Studente.
Precondizioni	L'utente ha selezionato un corso e ha la lista degli appunti di quel corso (4).
Flusso Principale	1. L'utente seleziona un appunto. 2. Il sistema recupera il testo (Markdown/Plain text) dal database. 3. Il sistema converte il testo in HTML e lo visualizza.
Postcondizioni	L'utente visualizza l'appunto renderizzato.

Tabella 4.6: Specifica del caso d'uso per RF9: CercaAppunto

ID	9
Nome	CercaAppunto
Descrizione	Permette la ricerca di appunti tramite titolo, contenuto o corso.
Attori	Studente.
Precondizioni	Il sistema è operativo.
Flusso Principale	1. L'utente inserisce una stringa nella funzione di ricerca. 2. Il sistema interroga il database cercando corrispondenze nei titoli. 3. Il sistema mostra i risultati filtrati.
Postcondizioni	L'utente seleziona un appunto dai risultati (prosegue con 5).

4.2 Dettagli del Caso d'Uso UC2

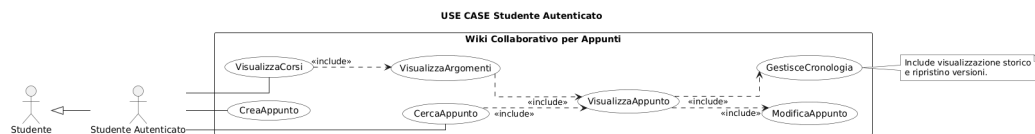


Figura 4.2: UC2 - Diagramma dei casi d'uso per l'attore Studente Autenticato

Tabella 4.7: Specifica del caso d'uso per RF6: CreaAppunto

ID	6
Nome	CreaAppunto
Descrizione	Permette a un utente autenticato di inserire un nuovo appunto nel sistema associandolo a un corso e a un argomento specifico.
Attori	Studente Autenticato.
Precondizioni	L'utente ha effettuato il login (2) e si trova nella sezione di creazione contenuti.
Flusso Principale	1. L'utente seleziona la funzione "Crea Appunto". 2. L'utente inserisce il titolo dell'appunto. 3. L'utente seleziona il corso e l'argomento di riferimento dal menu a tendina. 4. L'utente inserisce il contenuto testuale (Markdown o Plain Text). 5. L'utente conferma il salvataggio. 6. Il sistema valida i dati, memorizza l'appunto e associa l'ID dell'autore.
Postcondizioni	Il nuovo appunto è persistente nel database e visibile nella cronologia (RF8).
Flussi Alternativi	A1. Dati mancanti: Se il titolo o il contenuto sono vuoti, il sistema segnala l'errore e non procede al salvataggio.

Tabella 4.8: Specifica del caso d'uso per RF7: ModificaAppunto

ID	7
Nome	ModificaAppunto
Descrizione	Consente a un utente autenticato di modificare il testo di un appunto esistente. Ogni modifica genera automaticamente una nuova versione nella cronologia.
Attori	Studente Autenticato.
Precondizioni	L'utente è autenticato e ha visualizzato l'appunto che intende modificare.
Flusso Principale	1. L'utente ha completato lo UC5. 2. L'utente seleziona la funzione "Modifica". 3. Il sistema apre l'editor caricando il testo attuale dell'appunto. 4. L'utente apporta le modifiche desiderate. 5. L'utente conferma il salvataggio. 6. Il sistema salva le modifiche e genera automaticamente una nuova voce nella cronologia con timestamp e ID autore.
Postcondizioni	La versione corrente dell'appunto è aggiornata; la versione precedente è conservata nello storico (RF8).
Flussi Alternativi	A1. Annullamento: L'utente esce dall'editor senza salvare; il sistema non apporta modifiche e non crea nuove versioni.

Tabella 4.9: Specifica del caso d'uso per RF8: GestisceCronologia

ID	8
Nome	GestisceCronologia
Descrizione	Permette di visualizzare l'elenco delle versioni precedenti di un appunto e di effettuarne il ripristino.
Attori	Studente Autenticato.
Precondizioni	L'appunto selezionato ha subito almeno una modifica (esiste uno storico versioni).
Flusso Principale	1. L'utente ha completato lo UC5. 2. L'utente seleziona la voce "Cronologia versioni". 3. Il sistema mostra l'elenco delle versioni (timestamp e autore della modifica). 4. L'utente seleziona una versione precedente per visualizzarla. 5. L'utente richiede il ripristino della versione selezionata. 6. Il sistema sovrascrive la versione attuale con quella scelta, rendendola la più recente.
Postcondizioni	L'appunto viene riportato allo stato della versione selezionata.
Flussi Alternativi	A1. Sola consultazione: L'utente visualizza una versione passata ma non effettua il ripristino, tornando alla visualizzazione corrente.

4.3 Dettagli del Caso d'Uso UC3

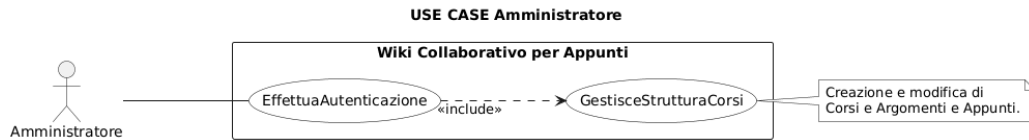


Figura 4.3: UC3 - Diagramma dei casi d'uso per l'attore Amministratore

Tabella 4.10: Specifica del caso d'uso per RF2: EffettuaAutenticazione (Admin)

ID	2
Nome	EffettuaAutenticazione
Descrizione	Consente all'Amministratore di accedere all'area gestionale del Wiki tramite credenziali riservate.
Attori	Amministratore.
Precondizioni	Il sistema è operativo e l'Amministratore possiede credenziali valide memorizzate tramite hashing.
Flusso Principale	1. L'Amministratore inserisce email e password nella pagina di login. 2. Il sistema invia i dati al backend tramite richiesta POST. 3. Il backend valida le credenziali confrontandole con i dati nel DB. 4. Il sistema avvia una sessione con privilegi elevati (Admin).
Postcondizioni	L'Amministratore è autenticato e viene reindirizzato alla dashboard di gestione corsi (11).
Flussi Alternativi	A1. Credenziali non valide: Il sistema nega l'accesso e restituisce un errore JSON (status 401 Unauthorized).

Tabella 4.11: Specifica del caso d'uso per RF11: GestisceStrutturaCorsi

ID	11
Nome	GestisceStrutturaCorsi
Descrizione	Permette all'Amministratore di creare, modificare o eliminare corsi e i relativi argomenti associati.
Attori	Amministratore.
Precondizioni	L'Amministratore ha effettuato con successo l'autenticazione (UC2).
Flusso Principale	1. L'Amministratore seleziona "Gestione Corsi" dalla dashboard. 2. Il sistema mostra l'elenco dei corsi esistenti recuperati dal DB. 3. L'Amministratore sceglie di inserire un nuovo corso o di modificarne uno esistente. 4. L'Amministratore inserisce/modifica il nome del corso e aggiunge gli argomenti correlati. 5. Il sistema invia i dati al backend tramite REST API (verbi POST/PUT). 6. Il backend aggiorna le tabelle relazionali nel database.
Postcondizioni	La nuova struttura dei corsi è salvata in modo persistente e visibile a tutti gli utenti (RF3, RF4).
Flussi Alternativi	A1. Validazione fallita: Se i campi obbligatori sono vuoti, il backend restituisce un errore validato tramite DTO. A2. Cancellazione Corso: L'Admin richiede l'eliminazione di un corso; il sistema verifica l'assenza di appunti collegati prima di procedere (integrità referenziale).

4.4 Matrice di tracciabilità

Tabella 4.12: Matrice di tracciabilità

Requisito Funzionale	UC1 (Stud)	UC2 (StudAuth)	UC3 (Admin)
RF1 (Registrazione)	X		
RF2 (Login/Logout)		X	X
RF3 (Lista corsi)	X		
RF4 (Appunti)	X		
RF5 (Visual. Appunto)	X		
RF6 (Crea appunti)		X	
RF7 (Modifica)		X	
RF8 (Cronologia)		X	
RF9 (Ricerca)	X		
RF10 (Gestione Admin)			X

Capitolo 5

Progettazione

5.1 Diagramma delle Classi FE

Il diagramma illustra l'architettura del client basata sul pattern **MVP (Model-View-Presenter)**, finalizzata alla separazione tra logica di presentazione e visualizzazione.

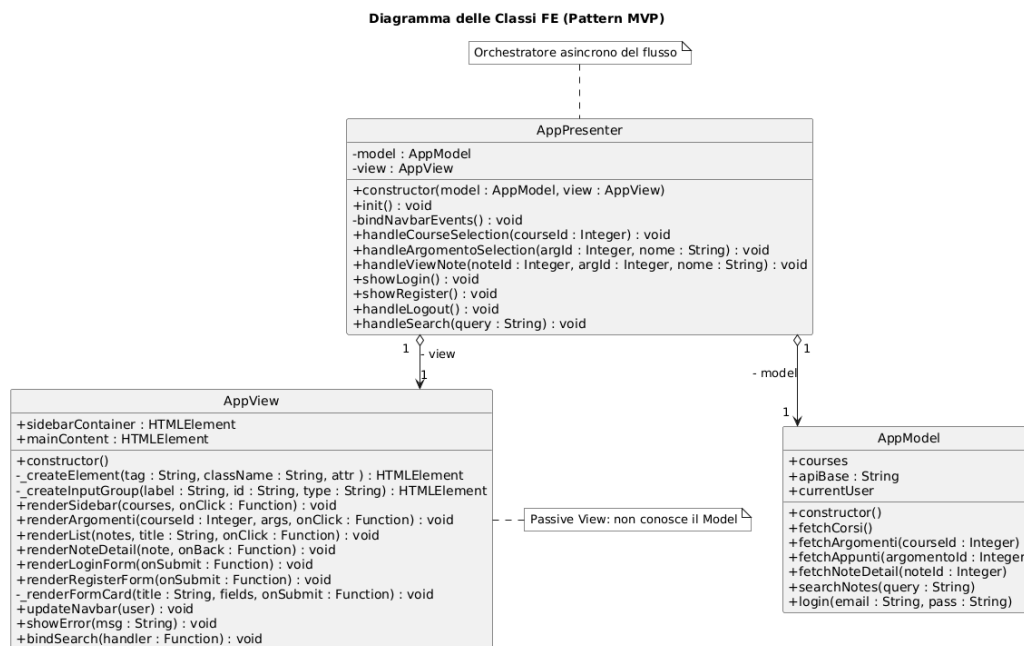


Figura 5.1: FE

5.2 Diagramma delle Classi BE

5.2.1 Diagramma delle Classi Entry Point

Il diagramma delle classi illustra la logica di *bootstrap* del backend, focalizzandosi sulla creazione delle risorse core e mostrando il collegamento con l'organizzazione del sottosistema di persistenza.

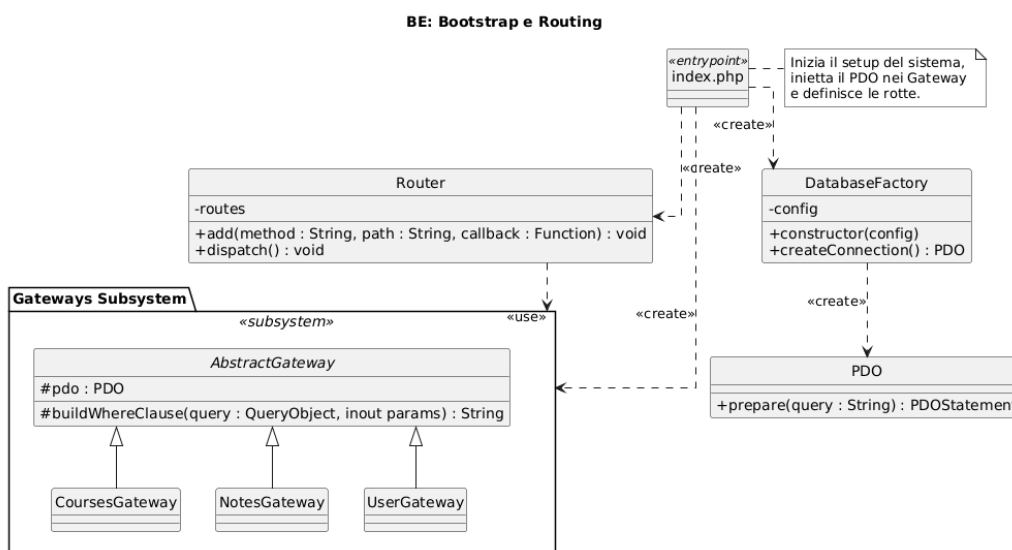


Figura 5.2: BE Entry Point

5.2.2 Diagramma delle Classi Gateway

Questo diagramma mostra la struttura fondamentale per l'accesso alle tabelle del database. Le classi Gateway incapsulano tutto il SQL necessario, rispettando il principio dell'Information Hiding.

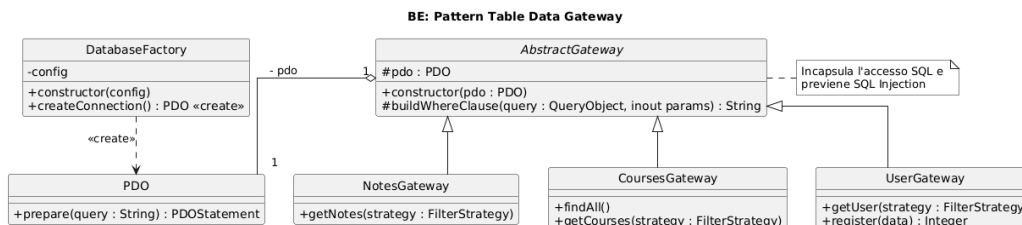


Figura 5.3: BE Gateway

5.2.3 Diagramma delle Classi Strategy

In questo diagramma viene rappresentata l'interazione tra il pattern Strategy e il Query Object. Questo approccio permette alle strategie di definire criteri logici di filtraggio (es. `corso_id = ?`) senza manipolare direttamente stringhe SQL grezze, risolvendo il problema dell'accoppiamento stretto presente nella versione originale del codice.

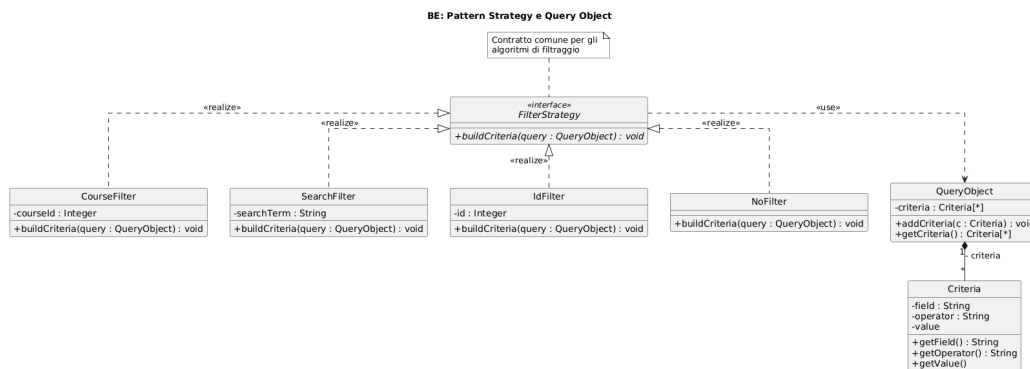


Figura 5.4: BE Strategy

5.3 Diagrammi di Sequenza

I seguenti diagrammi mostrano l'interazione temporale tra gli elementi dell'applicativo descrivendo alcuni casi d'uso.

5.3.1 Diagramma di Sequenza: Login (RF2)

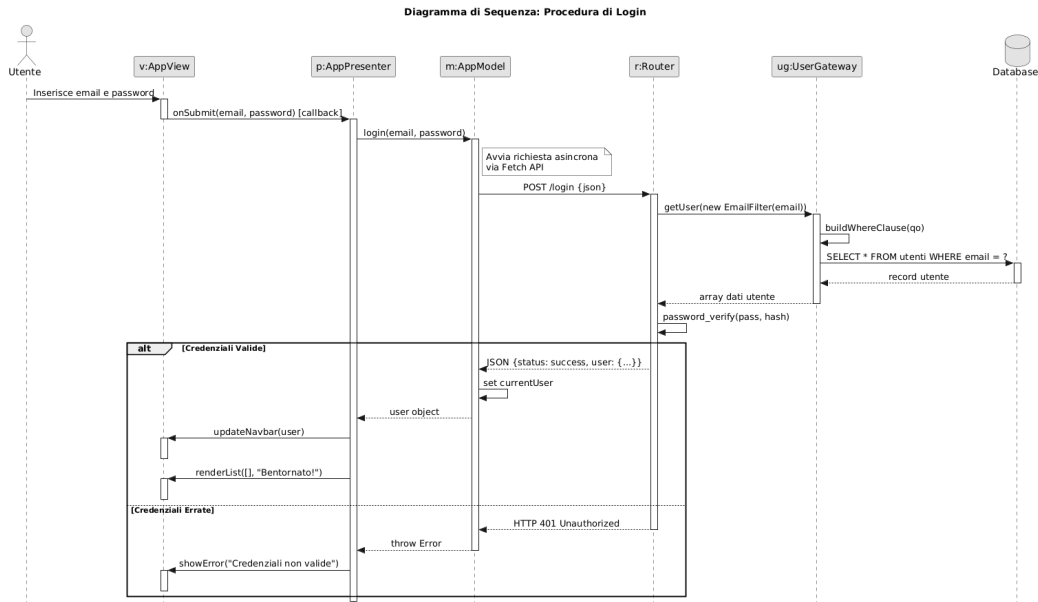


Figura 5.5: Login

Il processo è suddiviso nelle seguenti fasi:

- **Innesco nel Frontend:** La procedura ha origine nella *AppView*, la quale cattura l'evento di sottomissione (*onsubmit*) del modulo di accesso. In conformità con il paradigma della *Passive View*, la *View* non elabora i dati ma delega l'esecuzione al *AppPresenter* tramite una funzione di *callback*.
- **Richiesta Asincrona e Protocollo REST:** Il *Presenter* invoca il metodo *login(email, password)* del *AppModel*. Quest'ultimo utilizza la *Fetch API* per inoltrare una richiesta *POST stateless* all'endpoint */login*. Le credenziali vengono serializzate in formato **JSON** all'interno del corpo della richiesta, garantendo un interscambio di dati leggero e standardizzato.
- **Routing e Business Logic nel Backend:** Il *Router* riceve la richiesta e instrada il flusso verso lo *UserGateway*. Per recuperare le informazioni necessarie, viene istanziata una strategia concreta *EmailFilter* (Pattern **Strategy**). Questa popola un *QueryObject* con il criterio di uguaglianza sul campo *email*, che l'*AbstractGateway* trasforma in un *prepared statement SQL* sicuro per prevenire attacchi di *SQL Injection*.

- **Verifica di Sicurezza e Hashing:** Il *backend* non memorizza né confronta le password in chiaro. Il *Router* estrae l'*hash* dalla tabella *utenti* e utilizza la funzione nativa `password_verify()` per validare la stringa ricevuta dal *client*. Questo garantisce l'aderenza ai requisiti di riservatezza e sicurezza del sistema.
- **Gestione dello Stato e Risposta:**
 - **Caso di Successo:** Il server restituisce un oggetto **JSON** contenente i dati dell'utente (privato della password). Il `AppModel` aggiorna il proprio stato interno memorizzando l'utente in `this.currentUser`. Successivamente, il `Presenter` istruisce la `View` per aggiornare la barra di navigazione (`updateNavbar`) e mostrare un messaggio di benvenuto tramite `renderList`.
 - **Caso di Errore:** Qualora le credenziali siano errate, il server risponde con uno *status code* HTTP 401 `Unauthorized`. Il `Presenter` intercetta l'eccezione e invoca il metodo `showError` della `View` per informare l'utente senza esporre dettagli sensibili sulla natura del fallimento.

L'intero ciclo garantisce il **disaccoppiamento** tra la logica di persistenza e l'interfaccia utente, assicurando che ogni componente svolga esclusivamente i compiti previsti dal proprio ruolo architettonico.

5.3.2 Diagramma di Sequenza: Ricerca Appunti (RF9)

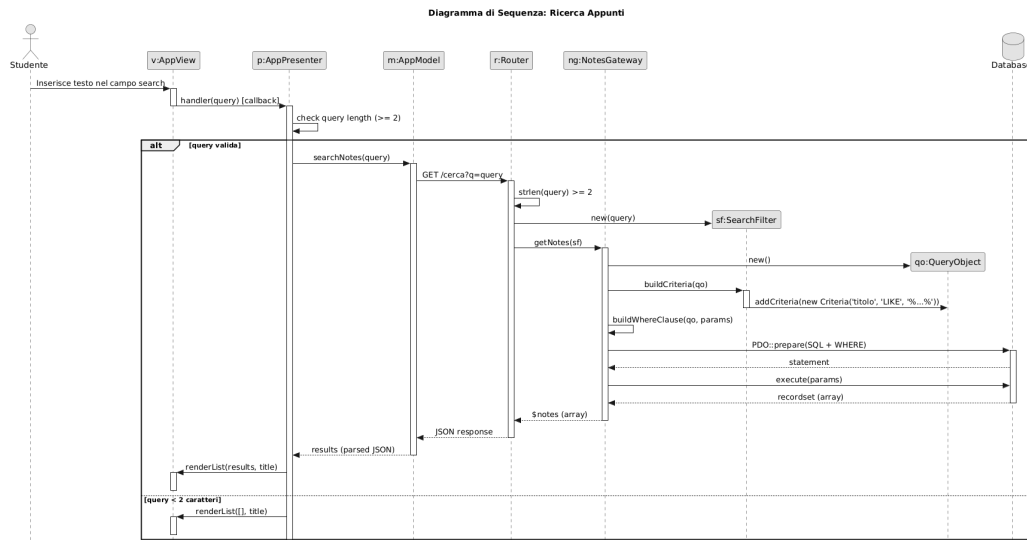


Figura 5.6: Ricerca Appunti

Il processo è suddiviso nelle seguenti fasi:

- Innesco e Validazione (Frontend):** Il processo ha origine nella **AppView**, la quale, tramite il metodo `bindSearch`, monitora l'input dell'utente sul **DOM**. Al verificarsi dell'evento, la **View** notifica l' **AppPresenter** tramite una funzione di *callback*. Il **Presenter** agisce come primo filtro logico, verificando che la lunghezza della stringa di ricerca sia pari o superiore a due caratteri prima di procedere.
- Comunicazione Asincrona e Routing:** In caso di validazione positiva, l' **AppPresenter** invoca il metodo `searchNotes` dell' **AppModel** [?]. Quest'ultimo esegue una chiamata asincrona utilizzando la **Fetch API** verso l'endpoint `/cerca?q=query`. Il **Router** del backend, ricevuta la richiesta, provvede a istanziare la strategia concreta **SearchFilter**, incapsulando il termine di ricerca all'interno dell'algoritmo di filtraggio.
- Elaborazione dei Criteri e Persistenza (Backend):** Il **NotesGateway** riceve l'oggetto **SearchFilter** nel metodo `getNotes`. Viene inizializzato un **QueryObject** che la strategia popola istruendolo a filtrare il campo `titolo` tramite l'operatore SQL `LIKE` e l'uso di caratteri jolly. Il metodo protetto `buildWhereClause` (ereditato da **AbstractGateway**) trasforma quindi i criteri astratti in una clausola `WHERE` parametrizzata,

garantendo la protezione contro *SQL Injection* tramite l'uso di *prepared statements* di PDO.

- **Risposta e Aggiornamento dell'Interfaccia:** Una volta recuperati i record dal database sotto forma di array associativo, il backend serializza i dati in formato **JSON** e li invia al client. L' **AppModel** riceve la risposta, esegue il *parsing* dei dati e li restituisce al **Presenter**. Infine, il **Presenter** istruisce la **View** affinché aggiorni lo stato visivo dell'applicazione richiamando il metodo `renderList`, completando così il ciclo di vita della richiesta.

5.4 Diagrammi di Attività

5.4.1 UC1 - Diagramma delle Attività: RF5 - Visualizzazione Dettaglio

Questo è specifico per RF5, ovvero l'apertura fisica del file Markdown e la sua conversione.

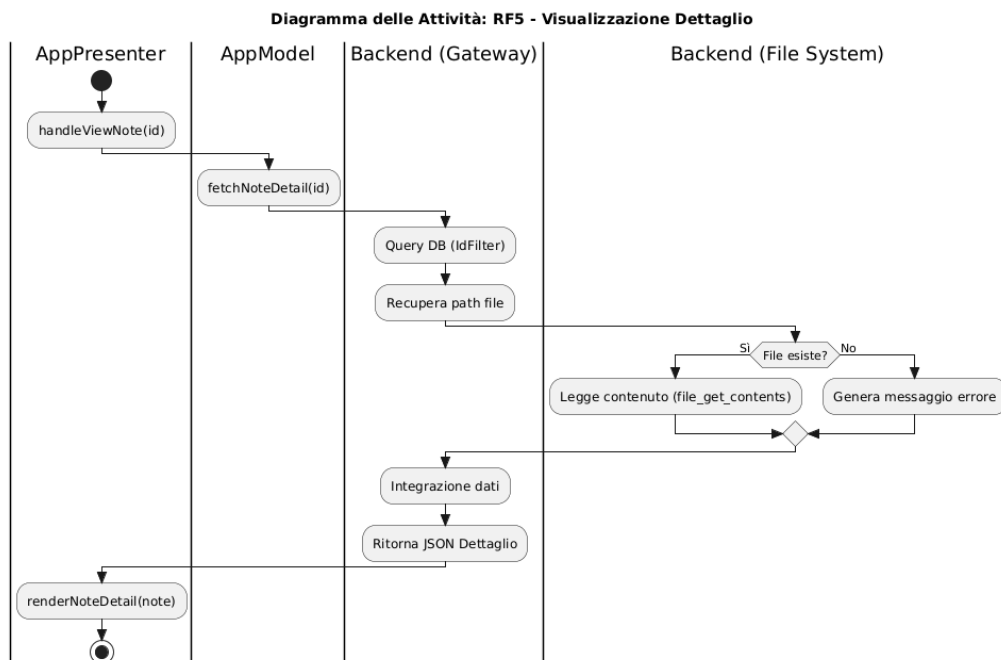


Figura 5.7: UC1 (RF5)

5.4.2 UC1 - Diagramma delle Attività: RF9 - Ricerca Appunti

Questo è specifico per RF9, ovvero la ricerca di un appunto.

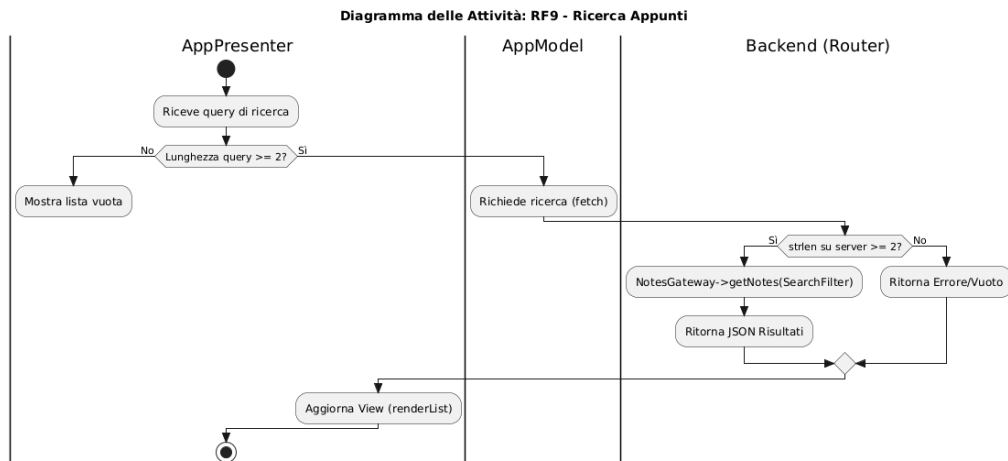


Figura 5.8: UC1 (RF9)

5.5 Progettazione dei Dati

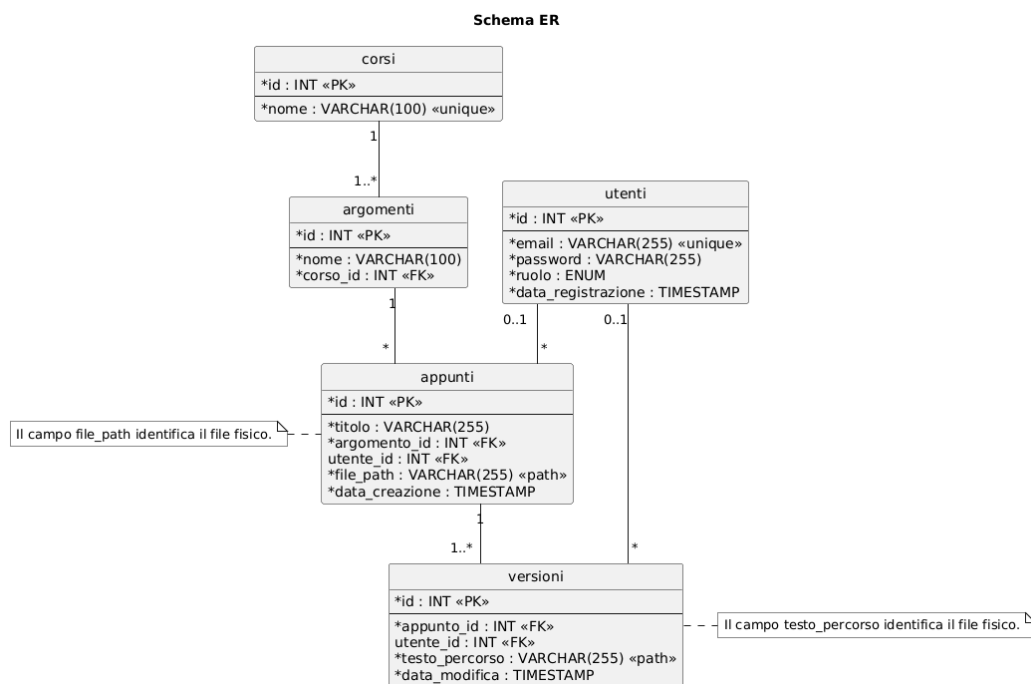


Figura 5.9: Schema Entity-Relationship

Diagramma delle classi DB ? (accennare salvataggio appunti non a DB)

5.6 Specifica REST API

La comunicazione tra il frontend (*JavaScript*) e il backend (*PHP*) del sistema Wiki Collaborativo avviene attraverso un'architettura di tipo **RESTful** (Representational State Transfer). Questa scelta garantisce un netto disaccoppiamento tra l'interfaccia utente e la logica di business, permettendo ai due componenti di evolvere indipendentemente e rendendo il sistema facilmente manutenibile e scalabile. Di seguito elenco i principi fondamentali del paradigma **REST**:

- **Risorse e URI:** Ogni entità del dominio, come *corsi*, *argomenti* e *appunti*, è trattata come una risorsa identificata univocamente da un **URI** (Uniform Resource Identifier) descrittivo e intuitivo, seguendo una struttura gerarchica coerente.

- **Protocollo Stateless:** Ogni richiesta inviata dal client al server è autocontenuta e include tutte le informazioni necessarie per l'elaborazione; al server non è richiesto di memorizzare lo stato del contesto dell'utente tra una chiamata e l'altra, favorendo la scalabilità del sistema.
- **Interfaccia Uniforme e CRUD:** Le interazioni avvengono tramite l'uso esplicito dei verbi standard **HTTP**, che mappano direttamente le operazioni **CRUD** (*Create, Read, Update, Delete*) sulle risorse: GET per il recupero (*Read*), POST per la creazione (*Create*), PUT per l'aggiornamento (*Update*) e DELETE per la rimozione (*Delete*).
- **Formato JSON:** Tutte le risposte prodotte dal backend sono strutturate in formato **JSON** (JavaScript Object Notation), garantendo un interscambio di dati leggero, facile da leggere per gli umani e semplice da interpretare per le macchine.

Questa interfaccia definisce il contratto formale tra client e server, basato sulla netta **Separation of Concerns**. Tale approccio assicura che il frontend possa essere teoricamente sostituito con tecnologie alternative (come applicazioni mobile o desktop) senza dover modificare la logica di persistenza e di business implementata nel backend.

5.6.1 Endpoint

Di seguito vengono dettagliati i principali endpoint.

1. Registrazione utente (RF1)

- **URL:** /register
- **Metodo:** POST
- **Parametri Query:**
 - email (obbligatorio).
 - password (obbligatorio).
- **Descrizione:** Riceve le credenziali dal frontend e crea un nuovo utente.

Esempio risposta di Successo (JSON):

```
{"status": "success", "id": 4}
```

2. Login (RF2)

- **URL:** /login
- **Metodo:** POST
- **Parametri Query:**
 - email (obbligatorio).
 - password (obbligatorio).
- **Descrizione:** Verifica la corrispondenza tra email e password hashata per avviare la sessione.

Esempio risposta di Successo (JSON):

```
{
  "status": "success",
  "user": {
    "id": 3,
    "email": "nico@studenti.unipr.it",
    "ruolo": "studente",
    "data_registrazione": "2026-01-31 18:18:59"
  }
}
```

3. Logout (RF2)

- **URL:** /logout
- **Metodo:** POST
- **Descrizione:** Invalida la sessione lato server e pulisce lo stato lato client.

4. Visualizzazione Corsi (RF3)

- **URL:** /corsi
- **Metodo:** GET
- **Descrizione:** Restituisce l'array di tutti i corsi presenti nel database.

Esempio risposta di Successo (JSON):

```
[
  { "id": 1, "nome": "Informatica" },
  { "id": 2, "nome": "Matematica" }
]
```

5. Visualizzazione argomenti di un corso (RF4)

- **URL:** /argomenti
- **Metodo:** GET
- **Parametri Query:**
 - **corso_id** (obbligatorio): Identificativo univoco del corso di riferimento. È un parametro obbligatorio affinché la richiesta vada a buon fine.
- **Descrizione:** Permette di recuperare l'elenco di tutti gli argomenti associati a un determinato corso selezionato dall'utente.

Esempio risposta di Successo (JSON):

```
[
  {
    "id": 4,
    "nome": "Meccanica",
    "corso_id": 3
  },
  {
    "id": 5,
    "nome": "Termodinamica",
    "corso_id": 3
  }
]
```

6. Recupero Appunti per Argomento (RF4,RF5)

- **URL:** /appunti
- **Metodo:** GET
- **Parametri Query:**
 - **argomento_id** (obbligatorio): Identificativo univoco dell'argomento scelto.

- **Descrizione:** Questo endpoint permette di ottenere la lista di tutti gli appunti associati a un determinato argomento. Risponde alla necessità dell'utente di navigare all'interno della gerarchia Corso → Argomento per giungere alla risorsa finale.

Esempio risposta di Successo (JSON):

```
[
  {
    "id": 4,
    "titolo": "Meccanica Classica",
    "argomento_id": 4,
    "utente_id": 1,
    "file_path": "storage\\notes\\3\\meccanica_classica.md",
    "data_creazione": "2026-01-30 14:53:36"
  },
  {
    "id": 5,
    "titolo": "Meccanica Statistica",
    "argomento_id": 4,
    "utente_id": 1,
    "file_path": "storage\\notes\\3\\meccanica_statistica.md",
    "data_creazione": "2026-01-30 14:53:36"
  }
]
```

7. Visualizzazione degli appunti (RF5)

- **URL:** /appunto
- **Metodo:** GET
- **Parametri Query:**
 - id (obbligatorio): Identificativo univoco dell'appunto scelto.
- **Descrizione:** Restituisce i metadati e il contenuto grezzo del file Markdown.

Esempio risposta di Successo (JSON):

```
{
  "id": 4,
```

```

    "titolo": "Meccanica Classica",
    "argomento_id": 4,
    "utente_id": 1,
    "file_path": "storage\\notes\\3\\meccanica_classica.md",
    "data_creazione": "2026-01-30 14:53:36",
    "contenuto": "# Meccanica classica\r\n\r\nLa **meccanica classica**[...]"
}

```

8. Ricerca (RF9)

- **URL:** /cerca
- **Metodo:** GET
- **Parametri Query:**
 - **q** (obbligatorio): Rappresenta il termine di ricerca inserito dall'utente. Il sistema backend processa la richiesta solo se la lunghezza di **q** è maggiore o uguale a 2 caratteri; in caso contrario, viene restituita una lista vuota per ottimizzare le prestazioni
- **Descrizione:** Permette di filtrare la collezione globale degli appunti cercando una corrispondenza testuale all'interno dei titoli.
- **Risposta di Successo (JSON):**

```

[
  {
    "id": 1,
    "titolo": "Introduzione al Corso",
    "argomento_id": 1,
    "utente_id": 1,
    "file_path": "storage\\notes\\1\\introduzione_al_corso.md",
    "data_creazione": "2026-01-30 14:53:36"
  },
  {
    "id": 6,
    "titolo": "Introduzione alla Termodinamica",
    "argomento_id": 5,
    "utente_id": 1,
    "file_path": "storage\\notes\\3\\introduzione_alla_termodinamica.md",
    "data_creazione": "2026-01-30 14:53:36"
  }
]

```

- 9. Creazione appunti (RF6)
- 10. Modifica appunti (RF7)
- 11. Lista degli ultimi aggiornamenti (RF10)
- 12. Gestione corsi, argomenti e appunti (RF11)
- 13. Cronologia delle versioni (RF8)

Capitolo 6

Architettura

6.1 Design Pattern Model-View-Presenter (MVP)

Il pattern MVP separa le responsabilità in tre componenti distinti: il **Model**, che incapsula la business logic e la persistenza; la **View**, che agisce come interfaccia passiva delegando ogni decisione al Presenter; e il **Presenter**, che funge da orchestratore. A differenza del pattern MVC, la View non osserva direttamente il Model, riducendo l'accoppiamento e facilitando il testing in isolamento.

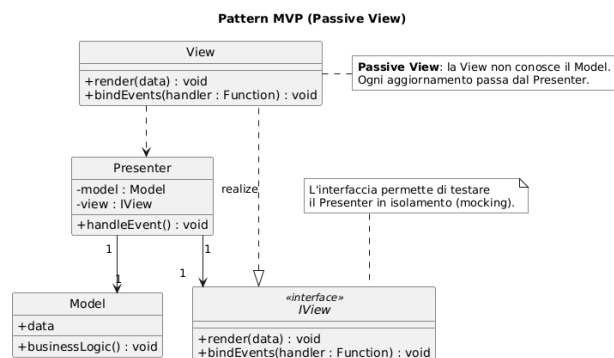


Figura 6.1: MVP

6.1.1 Mapping nel codice

- **AppPresenter (Presenter):** Funge da mediatore; riceve gli input dalla View e coordina le richieste verso il Model, aggiornando l'interfaccia in base ai dati ricevuti.
- **AppModel (Model):** Gestisce lo stato dei dati e l'interazione con le API REST del backend tramite richieste asincrone (*Promises*).
- **AppView (View e IView):** Rappresenta lo strato di visualizzazione (*Passive View*); si occupa esclusivamente della manipolazione del DOM e del rendering dei corsi e degli appunti.

6.2 Design Pattern Table Data Gateway

Il pattern **Table Data Gateway** fornisce un oggetto che funge da gateway verso una specifica tabella del database. Esso contiene tutta la logica SQL (*SELECT*, *INSERT*, *UPDATE*, *DELETE*) necessaria, isolando il resto dell'applicazione dai dettagli della persistenza. In genere restituisce un **Re-**

cordSet, ovvero una struttura dati generica che mima la natura tabulare dei dati.

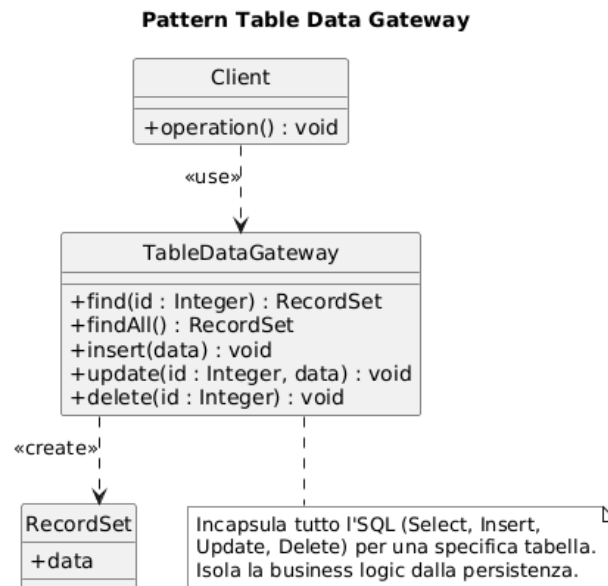


Figura 6.2: Table Data Gateway

6.2.1 Mapping nel codice

L'implementazione del sistema di persistenza segue una struttura gerarchica basata sul pattern **Table Data Gateway**, garantendo la separazione tra logica di business e accesso ai dati. Il mapping delle classi principali è articolato come segue:

- **AbstractGateway (Classe Base)**: Fornisce la logica comune a tutti i *gateway*. Centralizza il metodo protetto `buildWhereClause()` il quale è fondamentale per trasformare i criteri astratti, definiti tramite il pattern *Strategy*.
- **CoursesGateway**: Classe che gestisce la tabella `corsi`.
- **ArgomentiGateway**: Responsabile dell'interazione con la tabella `argomenti`.
- **NotesGateway**: Specializzazione architettónica deputata alla gestione esclusiva della tabella `appunti`.
- **UserGateway**: Gestisce la persistenza relativa alla tabella `utenti`.

6.3 Design Pattern Strategy

Il pattern **Strategy** è un pattern comportamentale utilizzato per definire una serie di algoritmi intercambiabili a runtime. Attraverso l'uso dell'aggregazione, il **Context** delega l'esecuzione del compito a un oggetto che realizza l'interfaccia **Strategy**.

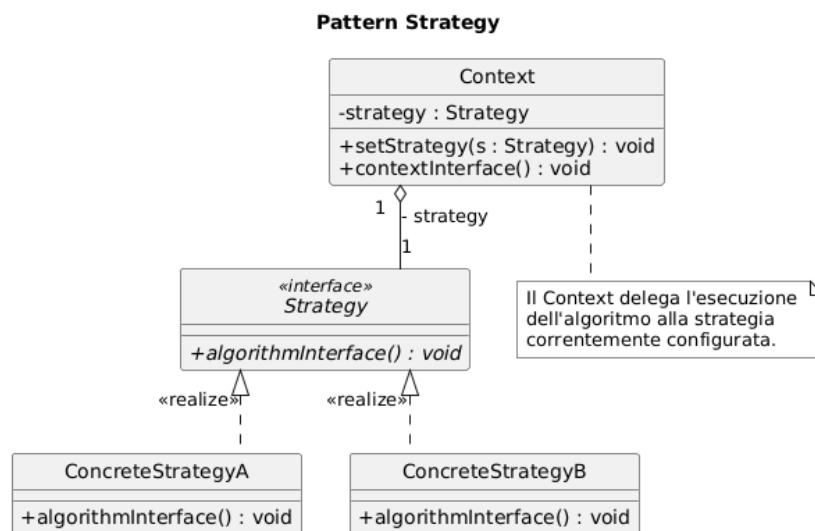


Figura 6.3: Strategy

6.3.1 Mapping nel codice

- **FilterStrategy (Interfaccia):** *Contratto* comune agli algoritmi di filtraggio. Definisce il metodo astratto `buildCriteria(QueryObject $query)`, che obbliga ogni strategia concreta a popolare un oggetto di query con i criteri specifici del dominio.
- **CourseFilter (Strategia Concreta):** Implementa il filtraggio degli argomenti in base all'identificativo di un corso specifico, in ottemperanza al requisito funzionale **RF4**.
- **ArgomentoFilter (Strategia Concreta):** Filtro degli appunti in base all'identificativo di un argomento specifico.
- **SearchFilter (Strategia Concreta):** Implementa la logica di ricerca testuale prevista dal requisito **RF9**. Istruisce il sistema a filtrare gli appunti il cui titolo corrisponde alla stringa inserita dall'utente tramite l'operatore SQL LIKE e l'utilizzo dei caratteri jolly %.

- **IdFilter** e **EmailFilter**: Ulteriori specializzazioni del pattern utilizzate rispettivamente per recuperare il dettaglio di un singolo appunto (**RF5**) o per estrarre le informazioni di un utente durante la procedura di *login* tramite il campo **email** memorizzato nel database.
- **NoFilter**: Utilizzata per soddisfare il contratto **Strategy** quando non c'è bisogno di fare alcun filtro.

6.4 Design Pattern Memento

L'architettura per la gestione dello stato e del ripristino è stata definita seguendo rigorosamente il design pattern **Memento**. Al fine di preservare l'incapsulamento, la classe **Memento** protegge lo stato interno dell'oggetto **Originator** esponendo al **Caretaker** un'interfaccia esclusivamente "stretta" (*narrow interface*), priva di accesso alla logica di business o alla struttura dei dati interna. Sotto il profilo strutturale, il legame tra **Originator** e **Memento** è formalizzato come una *Usage Dependency* caratterizzata dallo stereotipo normativo «create», che identifica la responsabilità dell'originatore nell'istanziamento dei checkpoint dello stato. Il coordinamento della cronologia è affidato al **Caretaker**, il quale mantiene una collezione di istanze mediante un'aggregazione condivisa (*shared aggregation*); l'uso della molteplicità [*] per la proprietà - **history** garantisce la gestione di una sequenza illimitata di stati, supportando operativamente funzionalità di *undo* a livelli multipli.

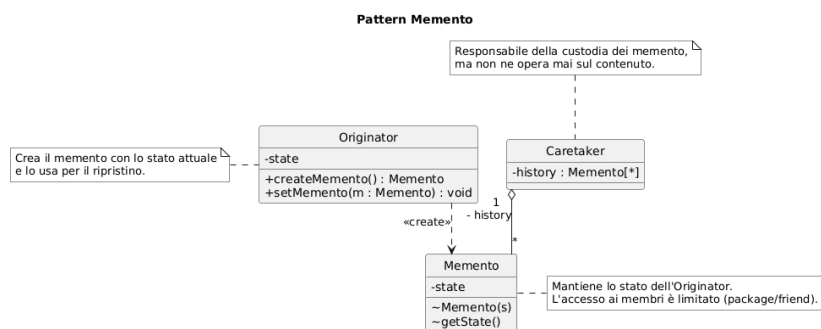


Figura 6.4: Memento

6.4.1 Mapping nel codice

Capitolo 7

Manuale Tecnico

7.1 Introduzione

Il sistema **Wiki UNIPR** è progettato per essere eseguito in un ambiente containerizzato tramite **Docker**. Questa architettura garantisce che il software funzioni in modo identico su qualsiasi macchina, isolando le dipendenze del Backend (PHP), del Database (MySQL) e del Frontend (Apache).

7.2 Requisiti di Sistema

Prima di procedere, assicurarsi che i seguenti strumenti siano installati e correttamente configurati:

- **Docker Desktop** (Windows/macOS) o **Docker Engine** (Linux).
- **Docker Compose** (solitamente incluso nelle versioni recenti di Docker).
- Un terminale (Bash, PowerShell o WSL).

7.3 Inizializzazione del Database

Il database viene configurato automaticamente durante il primo avvio del container DB.

7.3.1 Logica di Popolamento

Docker esegue gli script SQL presenti nella cartella `./services/database/configs/` seguendo l'ordine alfabetico dei nomi dei file. Per questo motivo, i file sono stati nominati con un prefisso numerico:

1. `01_init.sql`: Contiene i comandi `DROP DATABASE IF EXISTS` e `CREATE DATABASE` per garantire un ambiente pulito, seguiti dalla creazione delle tabelle con i relativi vincoli di integrità.
2. `02_dump.sql`: Contiene gli `INSERT` iniziali per popolare la tabella `corsi`, gli argomenti di test e gli utenti predefiniti.

7.4 Script di Automazione

Per semplificare la gestione dell'ambiente, sono stati predisposti tre script `.sh` nella root del progetto.

7.4.1 Avvio del Progetto (`run_project.sh`)

Utilizzare questo script per avviare l'intero stack. Lo script richiede all'utente se desidera effettuare un reset dei volumi per ricaricare il database da zero.

```
./run_project.sh
```

7.4.2 Arresto del Progetto (`stop_project.sh`)

Ferma i container in esecuzione senza eliminare i dati salvati nel database.

```
./stop_project.sh
```

7.4.3 Reset Totale (`reset_all.sh`)

Elimina container, volumi e immagini, ricostruendo l'ambiente da uno stato pulito. Utile in caso di modifiche strutturali al file `init.sql`.

```
./reset_all.sh
```

7.5 Procedura di Installazione Standard

1. Clonare il repository o estrarre i file del progetto.
2. Aprire il terminale nella cartella principale.
3. Fornire i permessi di esecuzione agli script:

```
chmod +x *.sh
```

4. Eseguire lo script di avvio:

```
./run_project.sh
```

5. Attendere circa 10-15 secondi per consentire a MySQL di completare l'inizializzazione.
6. Accedere all'app tramite browser all'indirizzo:

```
http://localhost:8080
```

7.6 Credenziali di Test

Per verificare le funzionalità del sistema (Login, consultazione e permessi), sono stati predisposti i seguenti account nel file di dump:

Tabella 7.1: Credenziali di test pre-caricate nel sistema.

Tipo Utente	Email	Password	Ruolo
Amministratore	admin@unipr.it	admin	amministratore
Studente	studente_test1@studenti.unipr.it	StudenteProva1	studente
Studente	studente_test2@studenti.unipr.it	StudenteProva2	studente
Studente	studente_test3@studenti.unipr.it	StudenteProva3	studente

7.7 Risoluzione dei Problemi

Se le tabelle non dovessero risultare presenti (*Table not found*), eseguire il comando `./reset_all.sh`. Questo forzerà Docker a cancellare il volume dati esistente e a rieseguire gli script di inizializzazione `01_init.sql` e `02_dump.sql`.